

Project Description

Wrapper classes for easy implementation of Tkinter menus.

This package was created to simplify the process of implementing menus in Tkinter windows. This was accomplished by defining functional "building block" classes which can be assembled together to create the user interface. Each of these classes provides the methods and properties needed to accomplish the assembly task. The design goal was to make this process as easy and intuitive as possible.

As a quick example, the following script will construct a Tkinter window and display a top menu bar that contains a single menu entry labeled **File**. This entry is a drop-down menu, and it has a selection item labeled **Exit**. Clicking on this item will call the **on_exit()** event handler, which closes the window and exits the program.

```
from tkinter import Tk
from menus import MainMenu

root = Tk()

def on_exit():
    root.destroy()

# Create and Populate the Top Menu Bar
menu_bar = MainMenu(root)
file_menu = menu_bar.add_menu('File')
file_menu.add_item('Exit', on_exit)

root.mainloop()
```

Installation

```
pip install easy-menus
```

Overview

This package provides the following class definitions :

- **MainMenu** - A class that displays a menu bar across the top of a window.
- **Menu** - A class used to represent a selection menu.
- **MenuItem** - A class used to represent a selection item in a Menu.
- **EntryType** - An enumerated dataclass of the available MenuItem entry types.
- **ConfigInfo** - A dataclass that provides MenuItem configuration information.
- **MenuButton** - A class used to represent a drop-down selection menu button.
- **ContextMenu** - A class used to represent a pop-up context menu.

The **MainMenu**, **Menu**, and **MenuItem** classes are the three "building blocks" used to construct the user interface for applications.

Consider the previous section's example script with the following changes :

1. The import statement now includes the **Menu**, and the **MenuItem** classes.
2. The **File** menu is first created and then added to the top menu bar.
3. The **Exit** selection item is first created and then added to the **File** menu.

```
from tkinter import Tk
from menus import MainMenu, Menu, MenuItem

root = Tk()

def on_exit():
    root.destroy()

# Create and Populate the Top Menu Bar
menu_bar = MainMenu(root)
file_menu = Menu('File')
menu_bar.add(file_menu)
exit_item = MenuItem('Exit', on_exit)
file_menu.add(exit_item)

root.mainloop()
```

Both scripts are functionally equivalent, but the second version shows the individual classes being created and then assembled to construct the user interface. The reader can refer to [The Zen of Python](#) to decide if either one of these two scripts is the more Pythonic than the other.

Referring back to the original example script, consider these two changes to the **Exit** selection item :

1. The **Exit** selection item's label string has been changed to '**E&xit**'
2. The **Exit** selection item's **shortcut** property has been assigned a key character value of '**w**'

```
from tkinter import Tk
from menus import MainMenu

root = Tk()

def on_exit():
    root.destroy()

# Create and Populate the Top Menu Bar
menu_bar = MainMenu(root)
file_menu = menu_bar.add_menu('File')
file_menu.add_item('E&xit', on_exit).shortcut = 'w'

root.mainloop()
```

The Windows and Linux platforms support the **Alt+Key** technique for navigating the top menu bar and its entries. The active key value for each entry appears as an underlined character in that entry's label. By default, the first character in the entry's label is the active key value. In this example, the top menu bar displays a **F**ile label. When the **&** symbol appears in the label string, the next character in the label will be designated as the active key value for that entry. The **&** symbol is not part of the displayed label. In this example, the '**E&xit**' string denotes that the **x** character is the active key value for this entry, and the label is displayed as **E**xit on the screen.

The **shortcut** property allows a selection item to have a **Control+Key (Command+key)** keyboard shortcut assigned to it. This is consistent with commonly used keyboard shortcuts such as the **Ctrl+C (⌘C)** shortcut for a **Copy**, or the **Ctrl+V (⌘V)** shortcut for a **Paste**. In this example, assigning the '**w**' character to the **shortcut** property creates a keyboard shortcut of **Ctrl+W (⌘W)** for the **Exit** selection item. The shortcut's name is displayed next to the label on the screen. The **MenuItem** class also has a **set_custom_shortcut()** method which can be used to assign other kinds of keyboard shortcuts, such as using a Function Key as a shortcut. The reader should refer to the Tkinter documentation for information on keyboard events.

The reader may wonder why the '**q**' character wasn't chosen for the **Exit** shortcut. The **Ctrl+Q** shortcut is commonly used to exit a program, and it certainly can be used on either a Windows or a Linux platform. However, the **⌘Q** shortcut is reserved for the system-level **Quit** command on macOS platforms. For the **⌘Q** shortcut to actually call the **on_exit()** event handler, the macOS user must override the system by adding the following statement :

```
root.createcommand('tk::mac::Quit', on_exit)
```

A menu entry can also display an icon image. Assuming there is an **image_folder** that contains an icon image file named **exit.png**, an icon image can be added to the **Exit** selection item by making the following changes to the previous example :

1. Import the PhotoImage class from tkinter
2. Create an **exit_icon** PhotoImage from the **image_folder/exit.png** image file
3. Add the **exit_icon** PhotoImage to the **Exit** selection item's argument list

```
from tkinter import Tk, PhotoImage
from menus import MainMenu

root = Tk()

def on_exit():
    root.destroy()

# Create and Populate the Top Menu Bar
menu_bar = MainMenu(root)
file_menu = menu_bar.add_menu('File')

exit_icon = PhotoImage(file='image_folder/exit.png')
file_menu.add_item('E&xit', on_exit, exit_icon).shortcut = 'w'

root.mainloop()
```

An application's top menu bar typically contains several drop-down menu entries. In this next example, four entries (**File**, **Edit**, **View**, and **Help**) are added to the top menu bar. The **begin_update()** method should be called prior to adding multiple entries to the **MainMenu** class. Calling this method causes all the added entries to be placed into a queue, and it prevents multiple screen updates from occurring during this process. When all the entries have been added, the **end_update()** method is called to process all the queued entries and to allow those entries to be displayed on the screen. Use of the **begin_update() ... end_update()** pair is recommended on all platforms, and it is required on the macOS platform to ensure the correct behavior of the top menu bar.

```
# Create and Populate the Top Menu Bar
menu_bar = MainMenu(root)
menu_bar.begin_update()
file_menu = menu_bar.add_menu('File')
edit_menu = menu_bar.add_menu('Edit')
view_menu = menu_bar.add_menu('View')
help_menu = menu_bar.add_menu('Help')
menu_bar.end_update()

...
```

Continuing this theme, a typical **File** drop-down menu will also have several selection items, such as **Open**, **Save**, and **Exit**. The `add_separator()` method displays a horizontal line between the **Save** and **Exit** selection items. Once again, the `begin_update()` ... `end_update()` pair should be used when adding multiple entries to an instance of the **Menu** class.

```
def on_open():
    print('File_Menu - Open')

def on_save():
    print('File_Menu - Save')

open_icon = PhotoImage(file='image_folder/open.png')
save_icon = PhotoImage(file='image_folder/save.png')
exit_icon = PhotoImage(file='image_folder/exit.png')

file_menu.begin_update()
file_menu.add_item('Open', on_open, open_icon).shortcut = 'o'
file_menu.add_item('Save', on_save, save_icon).shortcut = 's'
file_menu.add_separator()
file_menu.add_item('E&xit', on_exit, exit_icon).shortcut = 'w'
file_menu.end_update()

...
```

In this next code snippet, the **Edit** drop-down menu has the **Cut**, **Copy**, and **Paste** selection items. Here the **&** symbol appears in the **Cu&t** label string, and it is used to make that entry's active key value = **'t'**.

```
def on_cut():
    print('Edit_Menu - Cut')

def on_copy():
    print('Edit_Menu - Copy')

def on_paste():
    print('Edit_Menu - Paste')

edit_menu.begin_update()
edit_menu.add_item('Cu&t', on_cut).shortcut = 'x'
edit_menu.add_item('Copy', on_copy).shortcut = 'c'
edit_menu.add_item('Paste', on_paste).shortcut = 'v'
edit_menu.end_update()

...
```

The **View** drop-down menu has a **Zoom** menu entry, which in turn, has have three different zoom options. These three selection items are configured to behave like Tkinter Radiobutton widgets. First, the import statements from the previous example need to be updated to the following :

```
from tkinter import Tk, PhotoImage, IntVar
from menus import MainMenu, EntryType, ConfigInfo
```

Next, each selection item in the **Zoom** menu must be configured as a 'Radiobutton' entry. The **EntryType** and the **ConfigInfo** dataclasses are used to perform that task. The **EntryType** defines the behavior of the entry, and it can be one of three options : **STANDARD**(default), **CHECKBUTTON**, or **RADIOBUTTON**. Just like Radiobuttons, the Tkinter **IntVar** class is used to provide communication between the entries, and each entry must have a unique id value. The **ConfigInfo** dataclass is used to provide the configuration information when creating each of the three selection items. In this next code section, the three different zoom options are created and added to the **Zoom** menu entry in the **View** drop-down menu :

```
zoom_variable = IntVar(value=100)

def on_zoom():
    print(f'View_Menu - Zoom {zoom_variable.get()}%')

zoom_menu = view_menu.add_menu('Zoom')
zoom_menu.begin_update()
for value in (100, 200, 400):
    label = f'&{value}%'
    config = ConfigInfo(EntryType.RADIOBUTTON, zoom_variable, value)
    zoom_menu.add_item(label, on_zoom, config=config)
zoom_menu.end_update()

...
```

Finally, the **Help** drop-down menu has a single selection item labeled **About**, which has been assigned '**Ctrl+Shift+A**' as its custom keyboard shortcut.

```
def on_about():
    print('Help_Menu - About')

about = help_menu.add_item('About', on_about)
about.set_custom_shortcut('<Control-Shift-A>', 'Ctrl+Shift+A')

root.mainloop()
```

The **MenuButton** is essentially a single entry menu bar that can be positioned anywhere in the application window. The **MenuButton**'s text and optional image are always visible on the screen. Its drop-down menu, the **menu** property, is displayed when the **MenuButton** is clicked. This is an example using the **MenuButton** :

```
from tkinter import Tk
from menus import MenuButton

root = Tk()
selections = MenuButton(root, 'Selections', 20)
selections.grid(padx=40, pady=40)
selections.menu.begin_update()
for i in range(1, 5):
    def on_select(index=i):
        print(f'Selection Number {index}')
        selections.menu.add_item(f'Selection #{i}', on_select)
    selections.menu.add_item(f'Selection #{i}', on_select)
selections.menu.end_update()
root.mainloop()
```

The **ContextMenu** is a pop-up menu that can be displayed at a specified screen location. A **ContextMenu** is not visible until it is invoked by some action, usually a right-button mouse click. In this next example, a **ContextMenu** is created and then associated with a **Label** widget :

```
from tkinter import Tk, Label
from menus import ContextMenu

root = Tk()

def on_context_event():
    print('Context Menu Event')

context_menu = ContextMenu()
context_menu.add_item('Copy', on_context_event)
context_menu.add_item('Save As ...', on_context_event)
context_menu.add_item('Delete Text', on_context_event)

label = Label(root, text=' ContextMenu Example ', relief='groove')
label.grid(padx=40, pady=40)

def on_right_button(e): # Display the ContextMenu at the mouse position
    position = (label.winfo_rootx() + e.x, label.winfo_rooty() + e.y)
    context_menu.display(position)

label.bind('<Button-3>', on_right_button) # use '<Button-2>' on macOS
root.mainloop()
```

API Documentation

MainMenu

Displays a menu bar across the top of a window.

MainMenu(*parent*)

Construct and initialize the MainMenu.

- **parent : Any** - The application window displaying the MainMenu.

Properties

- **count : int** - The total number of items in the MainMenu. (readonly)
- **enabled : bool** - A value indicating whether the MainMenu is enabled. (read / write)

Methods

- **begin_update()** : Prevent screen redraws when updating the items in the MainMenu. This method should be called prior to making any updates or changes to the MainMenu.
- **end_update()** : Enable the screen to display the items in the MainMenu. This method should be called after the updates or changes to the MainMenu have been completed.
- **add(menu)** : Add a previously created Menu to the end of the MainMenu.
 - **menu : Menu** - The previously created Menu.
- **add_menu(text) -> Menu** : Create and add a Menu to the end of the MainMenu. This method returns the newly created Menu.
 - **text : str** - The text label string for the Menu.
- **add_range(menus)** : Add a list of previously created menus to the MainMenu.
 - **menus : list[Menu]** - The list of previously created Menus.

- **insert(*index*, *menu*)** : Insert a previously created Menu at the index location in the MainMenu.
 - ***index* : int** - The Menu's location within the MainMenu.
 - ***menu* : Menu** - The previously created Menu.
- **insert_menu(*index*, *text*) -> Menu** : Create and insert a Menu at the index location in the MainMenu. This method returns the newly created Menu.
 - ***index* : int** - The newly created Menu's location within the MainMenu.
 - ***text* : str** - The text label string for the Menu.
- **index_of(*menu*) -> int** : Report the index value of the specified Menu in the MainMenu. This method returns the zero-based index value if found, **-1** otherwise.
 - ***menu* : Menu** - The specified Menu in the MainMenu.
- **remove(*menu*)** : Remove the specified Menu from the MainMenu.
 - ***menu* : MenuItem | Menu** - The specified Menu.
- **remove_at(*index*)** : Remove a Menu from the MainMenu at the specified index location.
 - ***index* : int** - The specified Menu's location within the MainMenu.

Menu

A selection menu.

Menu(*text*, *image*=None)

Construct and initialize the Menu.

- **text : str** - The text label string for the Menu.
- **image : PhotoImage | None** - The optional image associated with the Menu, **default = None**.

Properties

- **count : int** - The total number of items in the Menu. (readonly)
- **enabled : bool** - A value indicating whether the Menu is enabled. (read / write)
- **image : PhotoImage | None** - The image associated with the Menu. (read / write)
- **text : str** - The text label string for the Menu. (read / write)
- **visible : bool** - A value indicating whether the Menu is visible. (read / write)

Methods

- **copy() -> Menu** : Create and return a deep copy of the Menu.
- **begin_update()** : Prevent screen redraws when updating the items in the Menu. This method should be called prior to making any updates or changes to the Menu.
- **end_update()** : Enable the screen to display the items in the Menu. This method should be called after the updates or changes to the Menu have been completed.
- **add(entry)** : Add a previously created entry to the end of the Menu.
 - **entry : MenuItem | Menu** - The previously created MenuItem or Menu entry.
- **add_item(text, command=None, image=None, config=None) -> MenuItem** : Create and add a MenuItem to the end of the Menu. This method returns the newly created MenuItem.
 - **text : str** - The text label string for the MenuItem.
 - **command : Callable | None** - The event handler associated with the MenuItem, **default = None**.
 - **image : PhotoImage | None** - The optional image associated with the MenuItem, **default = None**.
 - **config : ConfigInfo | None** - The optional MenuItem configuration information, **default = None**.
- **add_menu(text, image=None) -> Menu** : Create and add a Menu to the end of the Menu. This method returns the newly created Menu.
 - **text : str** - The text label string for the Menu.
 - **image : PhotoImage | None** - The optional image associated with the Menu, **default = None**.

- **add_range(*entries*)** : Add a list of previously created entries to the Menu.
 - **entries : list[MenuItem, Menu]** - The list of previously created MenuItem and Menu entries.
- **add_separator()** : Add a separator bar to the end of the Menu.
- **clear()** : Remove all existing entries from the Menu.
- **insert(*index*, *entry*)** : Insert a previously created entry at the index location in the Menu.
 - **index : int** - The entry's location within the Menu.
 - **entry : MenuItem | Menu** - The previously created MenuItem or Menu entry.
- **insert_item(*index*, *text*, *command*=None, *image*=None, *config*=None) -> MenuItem** : Create and insert a MenuItem at the index location in the Menu. This method returns the newly created MenuItem.
 - **index : int** - The newly created MenuItem's location within the Menu.
 - **text : str** - The text label string for the MenuItem.
 - **command : Callable | None** - The event handler associated with the MenuItem, **default = None.**
 - **image : PhotoImage | None** - The optional image associated with the MenuItem, **default = None.**
 - **config : ConfigInfo | None** - The optional MenuItem configuration information, **default = None.**
- **insert_menu(*index*, *text*, *image*=None) -> Menu** : Create and insert a Menu at the index location in the Menu. This method returns the newly created Menu.
 - **index : int** - The newly created Menu's location within the Menu.
 - **text : str** - The text label string for the Menu.
 - **image : PhotoImage | None** - The optional image associated with the Menu, **default = None.**
- **insert_separator(*index*)** : Insert a separator bar at the index location in the Menu.
 - **index : int** - The separator bar's location within the Menu.
- **index_of(*entry*) -> int** : Report the index value of the specified entry in the Menu. This method returns the zero-based index value if found, **-1** otherwise.
 - **entry : MenuItem | Menu** - The specified MenuItem or Menu entry in the Menu.
- **remove(*entry*)** : Remove the specified entry from the Menu.
 - **entry : MenuItem | Menu** - The specified MenuItem or Menu entry.
- **remove_at(*index*)** : Remove an entry from the Menu at the specified index location.
 - **index : int** - The entry's location within the Menu.

MenuItem

A selection item in a Menu.

`MenuItem( text, command=None, image=None, config=None )`

Construct and initialize the MenuItem.

- **text : str** - The text label string for the MenuItem.
- **command : Callable | None** - The event handler associated with the MenuItem, **default = None**.
- **image : PhotoImage | None** - The optional image associated with the MenuItem, **default = None**.
- **config : ConfigInfo | None** - The optional MenuItem configuration information, **default = None**.

Properties

- **checked : bool** - A value indicating whether a MenuItem is checked. (read / write)
- **enabled : bool** - A value indicating whether the MenuItem is enabled. (read / write)
- **image : PhotoImage | None** - The optional image associated with the MenuItem. (read / write)
- **shortcut : str** - The MenuItem's shortcut key character value. (read / write)
- **text : str** - The text label string for the MenuItem. (read / write)
- **visible : bool** - A value indicating whether the MenuItem is visible. (read / write)

Methods

- **copy() -> MenuItem** : Create and return a deep copy of the MenuItem.
- **set_custom_shortcut(sequence, accelerator)** : Set a custom, user-defined shortcut for the MenuItem.
The user should refer to the Tkinter documentation for information on keyboard events.
 - **sequence : str** - The custom event sequence string (e.g. '<Control-Shift-A>').
 - **accelerator : str** - The custom accelerator description (e.g. 'Ctrl+Shift+A').

EntryType

The available Menultem entry types.

- **STANDARD** - The Menultem entry is a standard menu selection.
- **CHECKBUTTON** - The Menultem entry behaves like a Tkinter Checkbutton widget.
- **RADIOBUTTON** - The Menultem entry behaves like a Tkinter Radiobutton widget.

ConfigInfo

The Menultem configuration information data class.

ConfigInfo(*entry_type*=EntryType.STANDARD, *variable*=None, *value*=0)

- ***entry_type* : EntryType** - The Menultem entry type identifier, **default = EntryType.STANDARD.**
- ***variable* : IntVar | None** - The Tkinter control variable associated with either a **CHECKBUTTON** or **RADIOBUTTON** Menultem entry, **default = None.**
- ***value* : int** - The control value assigned to a **RADIOBUTTON** Menultem entry, **default = 0.**

MenuButton

A drop-down selection menu button.

MenuButton(*parent*, *text*, *width*=-1, *image*=None)

Construct and initialize the MenuButton.

- **parent : Any** - The parent widget of the MenuButton
- **text : str** - The text label string for the MenuButton.
- **width : int** - The character width of the MenuButton, **default = -1 ('auto-size')**.
- **image : PhotoImage | None** - The optional image associated with the MenuButton, **default = None**.

Properties

- **enabled : bool** - A value indicating whether the MenuButton is enabled. (read / write)
- **image : PhotoImage | None** - The image associated with the MenuButton. (read / write)
- **text : str** - The text label string for the MenuButton. (read / write)
- **menu : Menu** - The MenuButton's drop-down menu. (readonly)

ContextMenu

A pop-up context menu.

ContextMenu()

Construct and initialize a ContextMenu.

Properties

The ContextMenu class is derived from the Menu class and inherits all the properties of that class.

Methods

The ContextMenu class is derived from the Menu class and inherits all the methods of that class.

Additional/Overridden Methods

- **copy() -> ContextMenu** : Create and return a deep copy of the ContextMenu.
- **display(location)** : Display the ContextMenu at the specified screen location.
 - **location : tuple[int, int]** - The specified screen location (x, y) of the upper left corner of the ContextMenu.

Usage Example

This example demonstrates a structured approach for creating the top menu bar of an application's main window class. Each of the **File**, **Edit**, and **View** drop-down menus is constructed by means of its own, individual class method and then added to the menu bar. This example also demonstrates an operating system independent implementation of the **Ctrl+Q** keyboard shortcut for exiting the application.

The **ImageList** class, from the [imagelist](#) package, is used to provide PhotoImages from the image files located in the specified '**image_folder**' (open.png, save.png, exit.png, cut.svg, copy.svg, paste.svg, and search.svg).

```
from tkinter import Tk, Frame, IntVar, filedialog, messagebox
import platform
from imagelist import ImageList
from menus import MainMenu, Menu, MenuItem, EntryType, ConfigInfo

class DemoForm(Frame):
    """The demonstration window."""

    def __init__(self, master: Tk, width: int, height: int):
        """Construct and initialize the demonstration window."""
        super().__init__(master, width=width, height=height)
        self.grid()
        self._master = master
        master.title('Demo Window')
        image_list = ImageList('image_folder', auto_load=True)

        # Create and Populate the Top Menu Bar
        menu_bar = MainMenu(self)
        menu_bar.begin_update()
        menu_bar.add(self._file_menu(image_list))
        menu_bar.add(self._edit_menu(image_list))
        menu_bar.add(self._view_menu())
        help_menu = menu_bar.add_menu('Help')
        about = help_menu.add_item('About', self._on_about)
        about.set_custom_shortcut('<F9>', 'F9')
        menu_bar.end_update()

    def _on_about(self):
        """Handle the 'About' selection."""
        self._console_print('Help_Menu - About')
```

This section of the application involves the creation of the **File** drop-down menu. The first group of statements in this section ensure that all the application window's exit/quit options are routed through the **self._on_exit** event handler. The first statement intercepts the close button in the window title bar. The second and third statements intercept the **⌘Q** keyboard shortcut on macOS systems. The final two statements construct the **Exit** selection item and assigns it the **Ctrl+Q** keyboard shortcut, regardless of the target platform (this appears as a **^Q** on a macOS platform).

The **Save** selection item and its keyboard shortcut are initially disabled. They will be enabled when the user chooses the **Open** selection item and selects an existing file in the dialog box.

```
def _file_menu(self, images) -> Menu:
    """Construct the 'File' drop-down menu."""
    self._master.protocol('WM_DELETE_WINDOW', self._on_exit)
    if platform.system() == 'Darwin': # macOS system
        self._master.createcommand('tk::mac::Quit', self._on_exit)
    exit_item = MenuItem('E&xit', self._on_exit, images['exit'])
    exit_item.set_custom_shortcut('<Control-q>', 'Ctrl+Q')

    menu = Menu('File')
    menu.begin_update()
    menu.add_item('Open', self._on_open, images['open']).shortcut = 'o'
    self._save_item = menu.add_item('Save', self._on_save, images['save'])
    self._save_item.shortcut = 's'
    self._save_item.enabled = False
    menu.add_separator()
    menu.add(exit_item)
    menu.end_update()
    return menu

def _on_open(self):
    """Handle the 'Open' selection."""
    filename = filedialog.askopenfilename(title='Open a File')
    if filename:
        self._save_item.enabled = True
        self._console_print(filename)

def _on_save(self):
    """Handle the 'Save' selection."""
    filename = filedialog.asksaveasfilename(title='Save a File')
    if filename:
        self._console_print(filename)

def _on_exit(self):
    """Handle the 'Exit' selection."""
    if messagebox.askyesno('Exit', 'Do you wish to Exit the program?'):
        self._master.destroy()
```


This next section of the application creates the **Edit** drop-down menu. Here, The **Paste** selection item and its keyboard shortcut are initially disabled. They will be enabled when the user chooses either the **Cut** or **Copy** selection item. A **Search** menu entry and its **Find** and **Replace** selection items have also been added to this drop-down menu.

```
def _edit_menu(self, images) -> Menu:
    """Construct the 'Edit' drop-down menu."""
    self._paste_item = MenuItem('Paste', self._on_paste, images['paste'])
    self._paste_item.shortcut = 'v'
    self._paste_item.enabled = False

    menu = Menu('Edit')
    menu.begin_update()
    menu.add_item('Cu&t', self._on_cut, images['cut']).shortcut = 'x'
    menu.add_item('Copy', self._on_copy, images['copy']).shortcut = 'c'
    menu.add(self._paste_item)
    menu.add_separator()
    search_menu = menu.add_menu('Search', images['search'])
    search_menu.begin_update()
    search_menu.add_item('Find', self._on_find).shortcut = 'f'
    search_menu.add_item('Replace', self._on_replace).shortcut = 'r'
    search_menu.end_update()
    menu.end_update()
    return menu

def _on_cut(self):
    """Handle the 'Cut' selection."""
    self._paste_item.enabled = True
    self._console_print('Edit_Menu - Cut')

def _on_copy(self):
    """Handle the 'Copy' selection."""
    self._paste_item.enabled = True
    self._console_print('Edit_Menu - Copy')

def _on_paste(self):
    """Handle the 'Paste' selection."""
    self._console_print('Edit_Menu - Paste')

def _on_find(self):
    """Handle the 'Find' selection."""
    self._console_print('Edit_Menu - Search_Menu - Find')

def _on_replace(self):
    """Handle the 'Replace' selection."""
    self._console_print('Edit_Menu - Search_Menu - Replace')
```

Finally, the **View** drop-down menu is created and populated with two different types of selection entries. The **Full Screen** selection item is configured as 'Checkbox' entry. Using its keyboard shortcut toggles the status (**True / False**) of the **checked** property. Once again, The **Zoom** menu entry has several 'RadioButton' selection entries.

The program begins execution at the **if __name__ == '__main__':** statement.

```
def _view_menu(self) -> Menu:
    """Construct the 'View' drop-down menu."""
    menu = Menu('View')
    menu.begin_update()
    self._full_screen = menu.add_item(
        'Full Screen',
        self._on_full_screen,
        config=ConfigInfo(EntryType.CHECKBUTTON),
    )
    self._full_screen.shortcut = 't' # Toggle the screen mode
    menu.add_separator()

    self._zoom_variable = IntVar(value=1)
    config = ConfigInfo(EntryType.RADIOBUTTON, self._zoom_variable)
    zoom_menu = menu.add_menu('Zoom')
    zoom_menu.begin_update()
    for value in (1, 2, 4, 8):
        config.value = value
        zoom_menu.add_item(f'&{value}00%', self._on_zoom, config=config)
    zoom_menu.end_update()
    menu.end_update()
    return menu

def _on_full_screen(self):
    """Handle the 'Full Screen' toggle selection."""
    self._master.attributes('-fullscreen', self._full_screen.checked)

def _on_zoom(self):
    """Handle the 'Zoom' selections."""
    self._console_print(f'View_Menu - Zoom {self._zoom_variable.get()}00%')

@staticmethod
def _console_print(message):
    """Print the message to the console."""
    print(f'*** {message} was Selected ***')

if __name__ == '__main__':
    main_form = DemoForm(Tk(), 400, 300)
    main_form.mainloop()
```